

Artificial Intelligence (AI-2002)

Lecture 5: The Structure of Agents

Mr. Saif ur Rehman (Lecturer – AI & DS)

saifurrehman@nu.edu.pk



The Structure of Agents

- Agent Behavior: Describes actions performed based on a sequence of percepts.
- Agent Program: Implements the agent function, mapping percepts to actions.
- Agent Architecture: The physical computing device with sensors and actuators.
- Agent = Architecture + Program.

Agent programs

function TABLE-DRIVEN-AGENT(*percept*) **returns** an action
 persistent: *percepts*, a sequence, initially empty
 table, a table of actions, indexed by percept sequences, initially fully specified

 append *percept* to the end of *percepts*
 action $\leftarrow$ LOOKUP(*percepts*, *table*)
 return *action*

Figure 2.7 The TABLE-DRIVEN-AGENT program is invoked for each new percept and returns an action each time. It retains the complete percept sequence in memory.

Agent programs

- For an agent, the lookup table grows exponentially based on percepts (P) and time (T).
- The table would contain $\sum_{t=1}^T |\mathcal{P}|^t$ entries.
- Consider the automated taxi: the visual input from a single camera comes in 30 frames per second, 1080×720 pixels with 24 bits of color information. This gives a lookup table with over entries $10^{600,000,000,000}$ for an hour's driving.

Agent programs

Challenges:

- Storage: No physical agent can store such large tables.
- Creation Time: Designers can't manually create such vast tables.
- Learning: An agent can't learn all the entries from experience.

Despite all this, TABLE-DRIVEN-AGENT does do what we want, assuming the table is filled in correctly: it implements the desired agent function.

Agent programs

Four Basic Types of Agent Programs:

1. Simple Reflex Agents
2. Model-Based Reflex Agents
3. Goal-Based Agents
4. Utility-Based Agents

These agents can be converted into learning agents to improve their performance over time.

Simple reflex agents

- Selects actions based only on the current percept, ignoring the history of past percepts.
- A vacuum agent whose decision is based solely on its current location and whether the location contains dirt.
- The action depends on the current percept, not on any previous ones.
- It does not retain or use past percepts to influence future decisions.
- Directly mapped from current percepts (e.g., location and dirt status).

Simple reflex agents

- A Simple Reflex Agent operates based on condition-action rules, where the agent's action is determined by the current percept (condition) without considering past percepts. The agent uses a rule like if Condition then Action, such as if location contains dirt then clean the location, to decide its behavior.

Simple reflex agents

```
function REFLEX-VACUUM-AGENT([location,status]) returns an action  
  if status = Dirty then return Suck  
  else if location = A then return Right  
  else if location = B then return Left
```

Figure 2.8 The agent program for a simple reflex agent in the two-location vacuum environment. This program implements the agent function tabulated in Figure 2.3.

Simple reflex agents

function SIMPLE-REFLEX-AGENT(*percept*) **returns** an action

persistent: *rules*, a set of condition–action rules

state $\leftarrow$ INTERPRET-INPUT(*percept*)

rule $\leftarrow$ RULE-MATCH(*state*, *rules*)

action $\leftarrow$ *rule*.ACTION

return *action*

Figure 2.10 A simple reflex agent. It acts according to a rule whose condition matches the current state, as defined by the percept.

Simple reflex agents

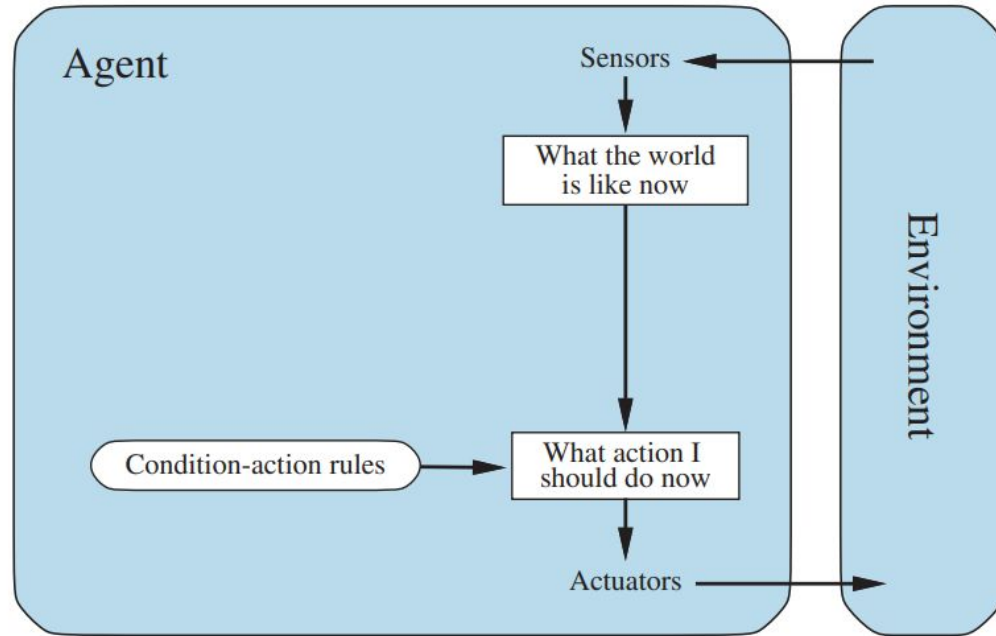


Figure 2.9 Schematic diagram of a simple reflex agent. We use rectangles to denote the current internal state of the agent's decision process, and ovals to represent the background information used in the process.

Write the Pseudocode for a Simple Reflex Agent:

The agent controls a door that should open or close based on motion detection and time of day.

Rules:

- Open the door if motion is detected and it's daytime.
- Close the door if no motion is detected and it's nighttime.

Model-based Reflex Agents

- The most effective way to handle partial observability is for the agent to keep track of the part of the world it can't see now. That is, the agent should maintain some sort of internal state that depends on the percept history and thereby reflects at least some of the unobserved aspects of the current state.

Model-based Reflex Agents

Updating Internal State: Requires two types of knowledge:

1. **Transition Model:** Describes how the world changes over time, divided into:
 - Effects of agent's actions (e.g., turning the steering wheel causes the car to turn right).
 - World evolution independently of the agent.
2. **Sensor Model:** Describes how the world state is reflected in the agent's percepts. E.g., braking car triggers red regions in the camera image; wet camera shows droplets.
3. **Model-Based Agent:** Combines transition and sensor models to track the state of the world, considering sensor limitations.

Model-based Reflex Agents

function MODEL-BASED-REFLEX-AGENT(*percept*) **returns** an action

persistent: *state*, the agent's current conception of the world state

transition_model, a description of how the next state depends on
the current state and action

sensor_model, a description of how the current world state is reflected
in the agent's percepts

rules, a set of condition–action rules

action, the most recent action, initially none

state $\leftarrow$ UPDATE-STATE(*state*, *action*, *percept*, *transition_model*, *sensor_model*)

rule $\leftarrow$ RULE-MATCH(*state*, *rules*)

action $\leftarrow$ *rule*.ACTION

return *action*

Figure 2.12 A model-based reflex agent. It keeps track of the current state of the world, using an internal model. It then chooses an action in the same way as the reflex agent.

Model-based Reflex Agents

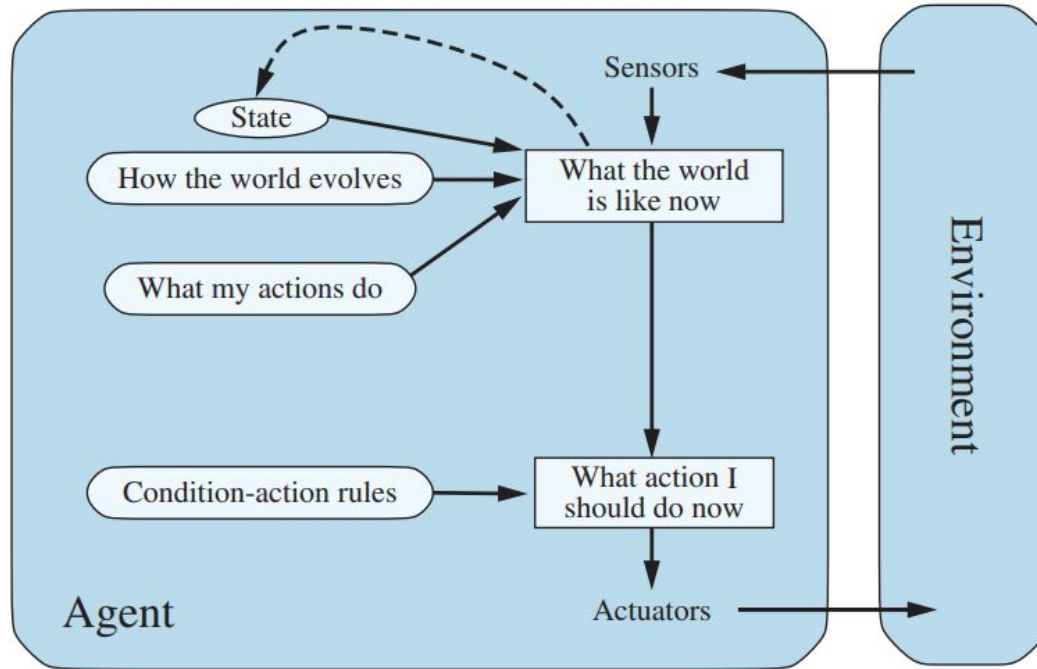


Figure 2.11 A model-based reflex agent.

Goal-based agents

- In some situations, knowing the current state alone isn't sufficient for decision-making (e.g., at a road junction, the taxi needs to decide which direction to take).
- The agent needs a goal that describes the desirable situation (e.g., reaching a particular destination). The goal is used to guide the agent's decisions and actions.
- Combines the current state description with goal information to make decisions that move the agent closer to achieving the goal.
- The goal provides a target or objective, helping the agent determine which action will best achieve the desired outcome

Goal-based agents

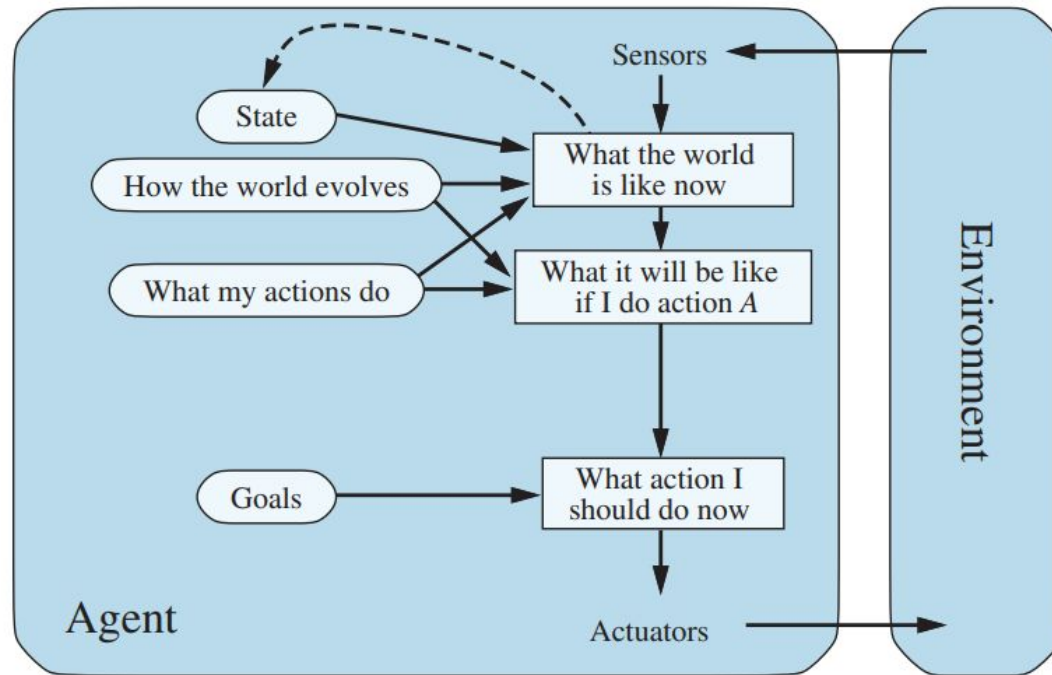


Figure 2.13 A model-based, goal-based agent. It keeps track of the world state as well as a set of goals it is trying to achieve, and chooses an action that will (eventually) lead to the achievement of its goals.

Utility-based agents

- Goals alone can't guarantee high-quality behavior, as multiple action sequences can achieve the goal, but some are better than others (e.g., quicker, safer, cheaper).
- Goals only distinguish between "happy" (goal achieved) and "unhappy" (goal not achieved) states, lacking granularity in how well the goal is achieved.
- Utility allows for a comparison between states, enabling the agent to make more informed choices, such as selecting a path that not only reaches the destination but also optimizes factors like cost, time, or safety.

Utility-based agents

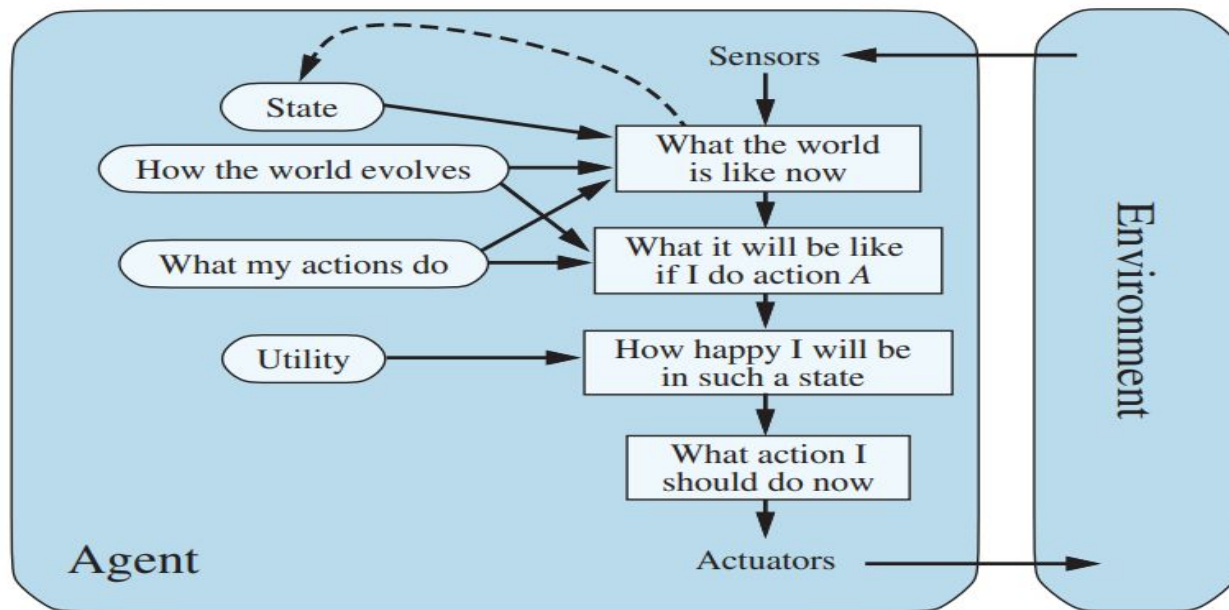


Figure 2.14 A model-based, utility-based agent. It uses a model of the world, along with a utility function that measures its preferences among states of the world. Then it chooses the action that leads to the best expected utility, where expected utility is computed by averaging over all possible outcome states, weighted by the probability of the outcome.

Learning agents

- Learning has another advantage, as we noted earlier: it allows the agent to operate in initially unknown environments and to become more competent than its initial knowledge alone might allow.

Learning agents

A learning agent consists of four main components: the performance element, the learning element, the critic, and the problem generator.

- Performance Element: This is the part of the agent that makes decisions based on the current percepts and selects actions to perform. It was previously considered the entire agent, as it is responsible for interacting with the environment.

Learning agents

- **Learning Element:** This component focuses on improving the agent's performance over time. It learns from experience and adjusts how the performance element operates based on feedback.
- **Critic:** The critic provides feedback on how well the agent is doing. It tells the learning element whether the performance is good or bad, helping the learning element understand what changes need to be made.

Learning agents

Problem Generator: The problem generator suggests new actions or situations for the agent to try, helping it explore and learn better strategies. It encourages the agent to experiment and expand its experience.

Learning agents

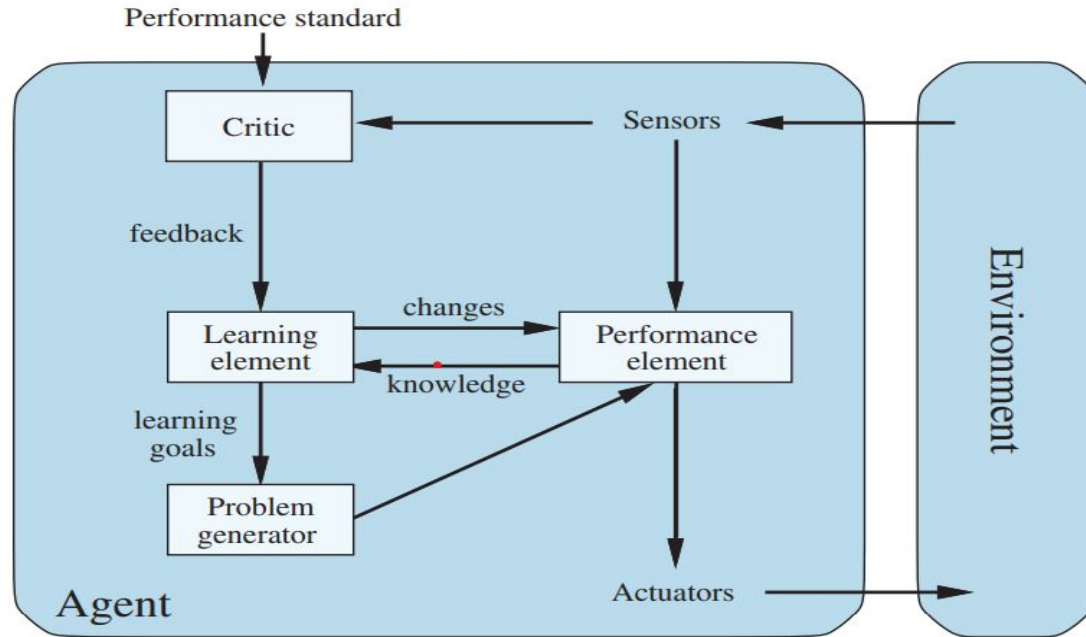


Figure 2.15 A general learning agent. The “performance element” box represents what we have previously considered to be the whole agent program. Now, the “learning element” box gets to modify that program to improve its performance.

1. You are in a smart home where the lights turn on automatically when you enter the room. The lights turn off when you leave, regardless of the time of day or current lighting conditions.
2. You are using a navigation app that continuously updates its route based on real-time traffic conditions and road closures.
3. A robot vacuum cleans your house by mapping the rooms. It avoids obstacles and navigates based on the information it has gathered about the layout of the house.
4. An online shopping assistant recommends products based on your preferences and previous purchase history, constantly adjusting its recommendations as it learns more about you.
5. A chess-playing program always selects the move that leads to a checkmate in the fewest possible moves.

1. Simple Reflex Agent
2. Model-Based Reflex Agent
3. Model-Based Reflex Agent
4. Learning Agent
5. Goal-Based Agent